Author: Tom Avital

Isracard - GPU Model Reporting System -- Work plan

Mission statement:

Objectives:

The entity being reported on is a "model". A "model" is any AI/ML workload that is running on nvidia GPU. The platform hosting the GPU and entities is OpenShift with nvidia GPUs (ranging from L40S, H100, B200). The platform has nvidia gpu operator and DCGM exporter alongside a plain grafana deployment (non-operator).

- List of models/entities
  - Configuration/allocation
- Report for utilization
  - configuration
  - allocation of resources
  - usage of resources
  - within pod - resource allocation like threads/pids
  - design and runtime configuration
- Report for resources
  - taken resources overall
  - Available resources
  - Free resources
- Report for "capacity"
  - Here "capacity" is defined as the growth rate of resource usage of different models over time - to see the trend for acquisition planning of future resources.

All reports should be snapshots of set time that can range:

- Hourly
- Daily
- Weekly
- Monthly

Phase 0: Foundation Validation

Flight: 1-3 Days

Goal: *Confirm* metrics availability, *establish* baseline queries

Tasks:

- Audit existing DCGM metrics in Prometheus -- verify metrics availability.
- Validate `kube-state-metrics` join capability to confirm GPU metrics correlation to pods/deployments/namespace
- Model definition documentation -- establish labeling convention to identify AI/ML workloads (namespace labels, pod annotations or naming conventions)

- Create metrics inventory: Map each objective to available Prometheus metric

Deliverables:

- Metrics availability matrix + "model" identification strategy.

Phase 1: Entity Inventory Dashboard

Flight: 1-3 days

Goal: *Build* a "List of models/entities" report

Objectives Covered:

- List of models/entities
- Configuration/allocation per model

Dashboard components:

Panel	Data Source	PromQL Pattern
Model inventory table	`kube-state-metrics` + `DCGM`	Join pod labels with GPU assignment
GPU type per model	`DCGM`	`DCGM_FI_DEV_GPU_TEMP{gpu_model=~".*"}` grouped by pod
Allocated vs requested GPUs	`kube-state-metrics`	`kube_pod_container_resource_requests{resource="nvidia_com_gpu"}`
MIG configuration	DCGM	MIG slice allocation per workload
Runtime config (threads/pids)	cAdvisor or custom exporter	Container process metrics

Tasks:

- Grafana dashboard layout design for entity inventory
- PromQL queries for each panel
- namespace/label filtering for model identification
- drill-down variables (namespace, GPU type, node pool, deployment labels)

Deliverables:

- "GPU Model Inventory" Grafana dashboard

Phase2: Utilization Reporting Dashboard

Flight: 2-5 days

Goal: Utilization report with multi-granularity.

Covers objectives:

- Resource usage (actual compute/memory/graphical/video/etc utilizatoin)
- Configuration vs actual usage comparison.
- Within-pod resource patterns

Dashboard components:

Panel	Metric	Time Aggregation
GPU compute utilization	`DCGM_FI_DEV_GPU_UTIL`	avg/max/p95 over time window
GPU memory utilization	`DCGM_FI_DEV_FB_USED / (FB_USED + FB_FREE)`	avg over time window
Unified load metric	Existing combined metric	Per-model breakdown
Idle detection	`DCGM_FI_DEV_GPU_UTIL < 10`	Duration/count
Over-provisioned models	Requested vs actual	Waste calculation

- **Time range support** - via dashboard variables for hourly/daily/weekly/monthly views using `$__range` and `avg_over_time()`

Tasks:

- Utilization dashboard with time-range selectors
- "Efficiency score" calculations (actual usage / allocated)
- Idle workload detection panel (Requires definition)
- Within-pod metrics if available (container level process stats)

Deliverable: "GPU Utilization Report" dashboard with snapshot export capability

Phase 3: Resource Availability Dashboard

Flight: 1-3 days

Goal: Build resource inventory report

Covers objectives:

- Taken resources overall
- Available resources
- Free resources

Dashboard components:

Panel	Calculation
Total GPU capacity	`sum(kube_node_status_capacity{resource="nvidia_com_gpu"})`
Allocated GPUs	`sum(kube_pod_container_resource_requests{resource="nvidia_com_gpu"})`
Free GPUs	Total - Allocated
GPU memory pool	Total FB across cluster
Memory in use	`sum(DCGM_FI_DEV_FB_USED)`
By GPU type breakdown	Group by L40S, H100, B200

Tasks:

- Resource pool overview dashboard
- Per-GPU-type capacity breakdown
- Node-level availability view
- Real-time vs allocated

Deliverables: "GPU Resource Pool" Dashboard

Phase 4: Capacity Planning Dashboard

Flight: 3-4 days

Covers objectives:

- Growth rate of resource usage
- Trend analysis for future resource planning

Dashboard components:

Panel	Analysis Type
Utilization trend (30/60/90 day)	`avg_over_time()` with increasing windows
Growth rate calculation	`deriv()` or `predict_linear()`
Peak usage patterns	Time-of-day/day-of-week heatmap
Capacity runway	"At current growth, X days until 80% capacity"
Per-model growth	Individual trend lines

Tasks:

- Long-term trend panels with linear prediction
- Growth rate calculations per model/namespace
- capacity runway projection
- Peak usage heatmap (identify busy periods for time-slot planning)
- Add seasonality detection if patterns exists (very very very ambitious and unrealistic)

Deliverable: "GPU Capacity planning" Dashboard

Phase 5: Automated Report generation

Flight: N/A

Report Types:

- Hourly: Brief operational status
- Daily: Full utilization summary + recommendations (unsure on how to handle recommendations yet)
- Weekly: trend analysis + capacity insights
- Monthly: ? overall summary

Implementation options:

- **Option A:** Grafana image renderer
  - Simplest
  - Configure scheduled PDF exports
  - Email delivery
- **Option B:** Python Cronjob (flexible)
  - Query Prometheus API directly
  - generate HTML/PDF reports with Jinja2
  - Custom recommendation logic can be implemented
  - OpenShift Cronjob deployment

Tasks:

- Report templates design for each interval
- Report generation mechanism implementation
- Delivery configuration
- Cronjob manifests for each interval
- Handle recommendation logic (recommendation engine - idle detection, migration candidates, capacity warnings)

Deliverable: Automated report pipeline & sample reports

Last phase Validation & refinement

Goal: Test with real data, gather feedback, iterate

Deliverable: Production-ready reporting system